



Airline Management System

Presenter:- Vaishnav verma ,
Anshu rana

Instructor:- Mr. Atul kumar

Introduction

What We Built

A relational database for an Airline Management System that handles:

-  Passenger registration
-  Flight scheduling
-  Ticket booking & seat allocation
-  Payment processing
-  Baggage tracking
-  Crew assignment

Our Goals

- Reduce data redundancy
- Ensure data accuracy
- Proper entity relationships
- Efficient data retrieval
- Scalable for future use

Entities & Attributes

Passenger

ID, Name, Age, Gender, Phone, Email,
Passport No

Booking

ID, Booking Date, Seat Number

Payment

ID, Amount, Payment Date, Method

Ticket

ID, Price, Ticket Type, Status

Flight

ID, Name, Departure Time, Arrival Time,
Seats

Baggage

ID, Weight, Type

Crew

ID, Name, Role, Contact No

Airport

ID, Airport Name, Location

Entity-Relationship Overview

Entity	Relationship	Related Entity	Cardinality
Passenger	— <i>makes</i> →	Booking	1 to Many
Booking	— <i>has</i> →	Payment	1 to 1
Booking	— <i>genrates</i> →	Ticket	1 to M
Booking	— <i>for</i> →	Flight	Many to 1
Passenger	— <i>carries</i> →	Baggage	1 to Many
Flight	— <i>has</i> →	Crew	1 to Many
Flight	— <i>departs from / arrives at</i> →	Airport	Many to 1

Normalization

Process of organizing a database to reduce redundancy and improve data integrity.

1NF — First Normal Form

Rule: Every column holds ONE atomic value. No repeating groups.

✗ Before: Flights_Booked: AI-101, AI-202 → ✓ After: Each flight = separate row in Booking table

2NF — Second Normal Form

Rule: No partial dependency. Non-key columns depend on the FULL primary key.

✗ Before: Flight_Name stored in Booking table (depends only on Flight_ID) → ✓ After: Flight details moved to a separate Flight table

3NF — Third Normal Form

Rule: No transitive dependency. Non-key column must not depend on another non-key column.

✗ Before: Airport_Name & Location stored inside Flight table → ✓ After: Airport info moved to a separate Airport table

Normalized Database Tables

Table Name	Primary Key	Foreign Keys
passengers	passenger_id	—
bookings	booking_id	passenger_id
payments	payment_id	booking_id
tickets	ticket_id	booking_id, flight_id
flights	flight_id	departure_airport_id, arrival_airport_id
baggage	baggage_id	passenger_id
crew	crew_id	flight_id
airports	airport_id	—

Conclusion

This project successfully demonstrates how a well-structured relational database can power a real-world Airline Management System. By applying normalization up to 3NF, we eliminated data redundancy, ensured data integrity, and created clear, logical relationships between all entities.

 8 normalized tables covering all aspects of airline operations

 All entities are connected using Primary & Foreign Keys

 1NF, 2NF, and 3NF achieved — no redundancy or bad dependencies

 The system is scalable, maintainable, and easy to query